Aufgabe

- Selektieren Sie alle Kurse deren `ModulName` mit P beginnt.

```
> select * from kurs where ModulName LIKE "P%";
+---------+-------------------------+------+-----------+
| ModulNr | ModulName               | CrP  | Dozent_id |
+---------+-------------------------+------+-----------+
| INF1003 | Programmieren 1         |    6 |        11 |
| INF1005 | Praktische Informatik 1 |    6 |        12 |
+---------+-------------------------+------+-----------+
2 rows in set (0.002 sec)
```

Aufgabe

Selektieren Sie alle Zeilen, in denen der ModulName mit „P“ beginnt und die Modulnummer auf 3 endet.

Lösung:

```
[pi2]> select * from kurs where ModulName like "P%" and ModulNr like "%3";
+---------+-----------------+------+-----------+
| ModulNr | ModulName       | CrP  | Dozent_id |
+---------+-----------------+------+-----------+
| INF1003 | Programmieren 1 |    6 |        11 |
+---------+-----------------+------+-----------+
1 row in set (0.001 sec)
```

Aufgabe

- Selektieren Sie alle Kurse von Dozent 12 oder mit einer `ModulNr` , die mit I beginnt.

```
> select * from kurs where dozent_id=12 or Modulnr like "I%";
+---------+-------------------------+------+-----------+
| ModulNr | ModulName               | CrP  | Dozent_id |
+---------+-------------------------+------+-----------+
| II1001  | Technische Informatik 1 |    6 |        13 |
| INF1003 | Programmieren 1         |    6 |        11 |
| INF1005 | Praktische Informatik 1 |    6 |        12 |
| TI5005  | Verteilte Algorithmen   |    6 |        12 |
+---------+-------------------------+------+-----------+
4 rows in set (0.003 sec)
```

Verbund mit WHERE

- Alle Kurse mit den Namen der Dozenten.

```
> select * from kurs, dozent where kurs.Dozent_id = dozent.id;
+---------+-------------------------+------+-----------+----+----------
| ModulNr | ModulName               | CrP  | Dozent_id | id | Name
+---------+-------------------------+------+-----------+----+----------+
| II1001  | Technische Informatik 1 |    6 |        13 | 13 | Bienhaus |
| INF1003 | Programmieren 1         |    6 |        11 | 11 | Herzberg |
| INF1005 | Praktische Informatik 1 |    6 |        12 | 12 | Meyer    |
| TI5005  | Verteilte Algorithmen   |    6 |        12 | 12 | Meyer    |
+---------+-------------------------+------+-----------+----+----------+
4 rows in set (0.001 sec)
```

Verbund mit WHERE und Umbenennung von Attributen

- Jetzt bitte ohne die `id`‘s und `DozentName` statt `Name`.

```
> select ModulNr, ModulName, Name as DozentName from kurs, dozent where kurs.Dozent_id = dozent.id;
+---------+-------------------------+------------+
| ModulNr | ModulName               | DozentName |
+---------+-------------------------+------------+
| II1001  | Technische Informatik 1 | Bienhaus   |
| INF1003 | Programmieren 1         | Herzberg   |
| INF1005 | Praktische Informatik 1 | Meyer      |
| TI5005  | Verteilte Algorithmen   | Meyer      |
+---------+-------------------------+------------+
4 rows in set (0.005 sec)
```

Sollte ein Attribut in mehreren der selektierten Tabellen vorkommen, schreibt man `<tabelle>.<attribut>`.



link

Verbund mit JOIN

- Effizienter und mächtiger als WHERE ist ein JOIN:

```
> select ModulNr, ModulName, Name from kurs k join dozent d on k.Dozent_id=d.id;
+---------+-------------------------+----------+
| ModulNr | ModulName               | Name     |
+---------+-------------------------+----------+
| II1001  | Technische Informatik 1 | Bienhaus |
| INF1003 | Programmieren 1         | Herzberg |
| INF1005 | Praktische Informatik 1 | Meyer    |
| TI5005  | Verteilte Algorithmen   | Meyer    |
+---------+-------------------------+----------+
4 rows in set (0.001 sec)
```

Die Namen der Tabellen kann man abkürzen, wenn gewünscht (z.B. `d.id`).
Natürlich sind Verbünde nicht auf zwei Tabellen beschränkt.

Left Join - 1

- Wir fügen einen Dozenten hinzu:
  `> insert into dozent values (14, „Vaupel“);`
- Alle Dozenten mit den Namen der Module, die sie halten.

```
> select Name, ModulName from dozent d join kurs  k  on d.id = k.dozent_id;
+----------+-------------------------+
| Name     | ModulName               |
+----------+-------------------------+
| Bienhaus | Technische Informatik 1 |
| Herzberg | Programmieren 1         |
| Meyer    | Praktische Informatik 1 |
| Meyer    | Verteilte Algorithmen   |
+----------+-------------------------+
4 rows in set (0.001 sec)
```

Wo ist der neue Dozent?

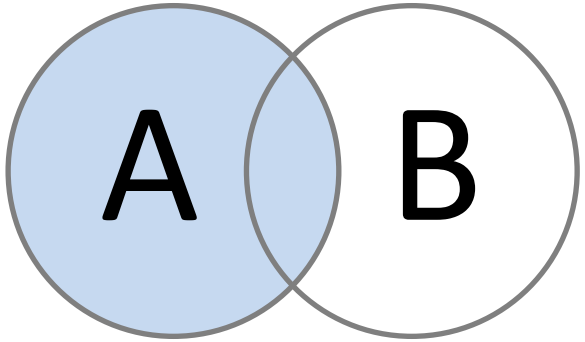
Left Join - 2

- Da es keinen `kurs` gibt, bei dem die `dozenten_id = 14` ist, wird der Dozent Vaupel nicht angezeigt.
- Statt des `JOIN` muss dann ein `LEFT JOIN` verwendet werden:

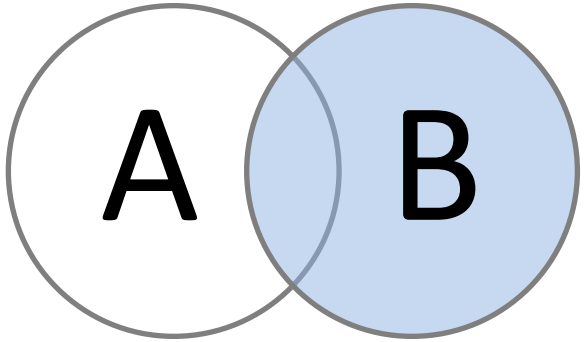
```
> select Name, ModulName from dozent d left join kurs  k  on d.id = k.dozent_id;
+----------+-------------------------+
| Name     | ModulName               |
+----------+-------------------------+
| Herzberg | Programmieren 1         |
| Meyer    | Praktische Informatik 1 |
| Meyer    | Verteilte Algorithmen   |
| Bienhaus | Technische Informatik 1 |
| Vaupel   | NULL                    |
+----------+-------------------------+
5 rows in set (0.001 sec)
```

`RIGHT JOIN` ist entsprechend.

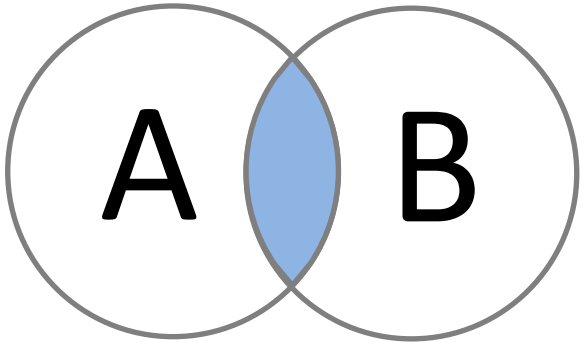
JOINS



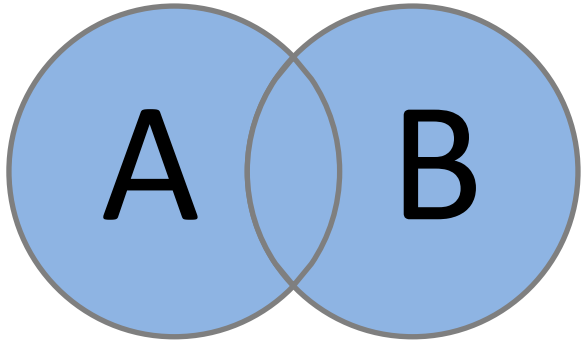
LEFT JOIN



Right JOIN



Inner JOIN (= JOIN)



FULL OUTER JOIN

JOINS

Tabelle A

Id	Name
1	a
2	b
3	c

Tabelle B

Id	Name
4	x
5	y
6	z

MariaDB unterstützt full outer join nicht!

INNER JOIN: select a.name, b.name from A join b on a.id=b.id;

Name	Name

LEFT JOIN: select a.name, b.name from A left join b on a.id=b.id;

Name	Name
a	NULL
b	NULL
c	NULL

RIGHT JOIN: select a.name, b.name from A right join b on a.id=b.id;

Name	Name
NULL	x
NULL	y
NULL	z

Aufgabe

- Drehen Sie die Reihenfolge der Tabellen in der vorigen Abfrage um, sodass `kurs` mit `dozent` durch einen `left join` verbunden wird.

```
>
```

Welchen Join müssen Sie nun verwenden, um das gleiche Ergebnis wie beim left join zu erreichen?

Mehr SELECT-Optionen

- `DISTINCT`
  Entfernt doppelte Datensätze aus der Antwort.
- `ORDER BY`
  Sortiert das Ergebnis nach den Werten der angegebenen Attribute
  - `ASC` aufsteigend
  - `DESC` absteigend

Aggregate

- Wie viele Kurse gibt es?

```
> select count(*) from kurs;
+----------+
| count(*) |
+----------+
|        4 |
+----------+
1 row in set (0.003 sec)
```

- Anzahl der Dozenten in Tabelle kurs?

```
> select count(dozent_id) from kurs;
+------------------+
| count(dozent_id) |
+------------------+
|                4 |
+------------------+
1 row in set (0.001 sec)
```

- Besser:

```
select count(distinct dozent_id) from kurs;
+---------------------------+
| count(distinct dozent_id) |
+---------------------------+
|                         3 |
+---------------------------+
1 row in set (0.001 sec)
```

GROUP BY

- Neben `COUNT` gibt es noch die Funktionen `SUM`, `MIN`, `MAX`, `AVG`.
- Aggregate werden oft in Verbindung mit `GROUP BY` verwendet:

```
select name, count(*) from kurs k, dozent d where k.dozent_id=d.id group by d.id;
```

```
+----------+----------+
| name     | count(*) |
+----------+----------+
| Herzberg |        1 |
| Meyer    |        2 |
| Bienhaus |        1 |
+----------+----------+
3 rows in set (0.001 sec)
```

Aufgabe

- Ermitteln Sie die durchschnittliche Punktzahl

a) Pro Studentin/Student

b) Pro Kurs

Lösung:

a)

b)

```
select * from ratings;
+---------+---------+--------+
| sid     | kid     | rating |
+---------+---------+--------+
| 5551111 | II1001  |     80 |
| 5551111 | INF1003 |     70 |
| 5552222 | INF1003 |     60 |
| 5552222 | INF1005 |     75 |
| 5553344 | INF1005 |     95 |
| 5555566 | TI5005  |     80 |
+---------+---------+--------+
```

GROUP BY und HAVING

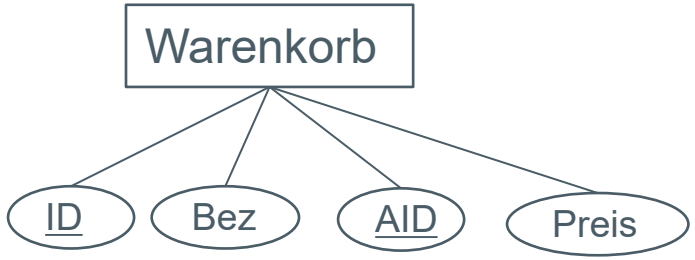
- Möchte man die aggregierten Werte in Bedingungen einbeziehen, muss man diese hinter HAVING aufführen (nicht nach WHERE!).

```
select name, count(*) from kurs k, dozent d
       where k.dozent_id=d.id
       group by d.id
       having count(*)>1;
```

```
+----------+----------+
| name     | count(*) |
+----------+----------+
| Meyer    |        2 |
+----------+----------+
3 rows in set (0.001 sec)
```

Aufgabe

Aufgabe: Erstellen Sie eine SQL-Abfrage, die die Gesamtkosten jedes einzelnen Warenkorbs ermittelt.




```
select * from warenkorb1;
+------+------------+------+--------+
| id   | bez        | aid  | preis  |
+------+------------+------+--------+
|    1 | Ostern     |   34 | 149.00 |
|    1 | Ostern     |   42 |  99.00 |
|    2 | Geburtstag |   42 |  99.00 |
+------+------------+------+--------+
3 rows in set (0.001 sec)
```

Index (Plural: Indexe)

- Ein Index in einer Datenbank ist eine Datenstruktur, die die Geschwindigkeit von Datenabrufoperationen auf einer Datenbanktabelle verbessert.
- Er funktioniert wie eine organisierte Liste von Schlüsselwerten, die auf die genaue Position von Daten innerhalb der Tabelle verweisen.
- Durch das Erstellen eines Index auf einer oder mehreren Spalten kann die Datenbank schnell die Zeilen finden und darauf zugreifen, die den Abfragebedingungen entsprechen, was die gesamte Abfrageverarbeitungszeit reduziert.
- Genauso wie ein Index in einem Buch Ihnen hilft, schnell Informationen zu finden, beschleunigt ein Datenbankindex den Datenzugriff.
- Wird zum Beispiel oft nach „Name“ in der Artikel-Tabelle gesucht erzeugt man einen Index mit:

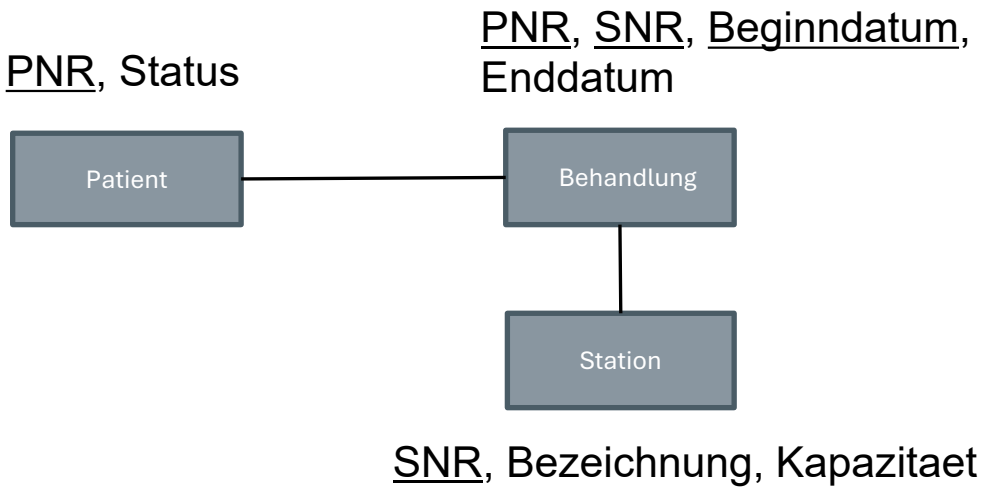
```
> CREATE INDEX idx_name ON Artikel(Name);
```



Konsistenz

Unter Datenkonsistenz versteht man die Widerspruchsfreiheit der Daten zu sich selbst und zu den Vereinbarungen des konzeptionellen Modells.

Beispiele:



Welche Konsistenzbedingungen würden Sie erwarten?

Transaktion

Eine **Transaktion** ist eine Folge zusammenhängender DB-Operationen, bei deren Ausführung auf einer konsistenten DB die Konsistenz der DB erhalten bleibt.

Eine Transaktion überführt eine Datenbank von einem konsistenten Zustand in einen anderen konsistenten Zustand.

Eine Transaktion besteht aus:

START TRANSACTION
SQL Befehle
COMMIT** oder **ROLLBACK

- COMMIT: Übernahme der Daten in die DB
- ROLLBACK: Abbruch der Transaktion. „Zurückrollen“ aller Änderungen

Beispiel

```
> create table buchungen (id int primary key, typ text, kundenID int, preis numeric(5,2));
```

D.h. maximaler Preis pro Buchung 999.99€.

Kunde 4711 möchte nun Hotel und Flug buchen – aber nur wenn beide buchbar sind.

```
> insert into buchungen values (1, "Hotel", 4711, 125.00);
> insert into buchungen values (2, "Flug", 4711, 1050);
ERROR 1264 (22003): Out of range value for column 'preis' at row 1
MariaDB [pi2]> select * from buchungen;
+----+-------+----------+--------+
| id | typ   | kundenID | preis  |
+----+-------+----------+--------+
|  1 | Hotel |     4711 | 125.00 |
+----+-------+----------+--------+
```

Und nun?

Lösung

```
> start transaction;
> insert into buchungen values (1, "Hotel", 4711, 125.00);
> insert into buchungen values (2, "Flug", 4711, 1050);
ERROR 1264 (22003): Out of range value for column 'preis' at row 1
> rollback;
> select * from buchungen;
Empty set (0.000 sec)
```

ACID

Transaktionen sollen nach dem **ACID-Prinzip** abgearbeitet werden:

Atomicity
: Transaktionen sind elementar, nicht unterbrechbar, d.h., sie werden vollständig oder gar nicht ausgeführt. *(„Alles-oder-Nichts-Prinzip“)*

Consistency
: Transaktionen sind konsistenzerhaltend.

Isolation
: Benötigte Datenobjekte sind vor anderen Operationen geschützt. Das Ergebnis einer Transaktion wird nicht durch parallel ablaufende Transaktionen beeinflusst.

Durability
: Wirkung einer Transaktion ist dauerhaft.

Das Akronym ACID geht auf eine Publikation der beiden Kaiserslauterner Informatiker Theo Härder und Andreas Reuter zurück.